PiStorm32Lite I2C LCD Display
Windows WSL Development Setup Guide

From a fresh Windows computer to compiling pistormlcd_integrated
Beginner edition | 17 October 2026 | Michael Gruber / Sydney - Australia

Repository: github.com/mkeguber/Amiga-PiStorm32Lite-I2C-LCD-Display

What this guide covers

This guide sets up Windows Subsystem for Linux (WSL), Ubuntu, development utilities and an AmigaOS 68k cross-compiler. It then shows how to retrieve the PiStorm LCD source, identify its required headers, build the known working integrated program, and use an AI assistant to make careful changes. No previous Linux experience is assumed.

Scope and accuracy: The final working environment and compile command are documented from the project development history. Some original installation commands (especially the exact cross-compiler distribution and the provenance of custom headers) were not preserved. Where necessary, this guide supplies a reproducible modern setup rather than claiming to reconstruct every historical command verbatim. GitHub source-file names under `src/` could not be enumerated reliably during publication; commands below discover them locally after cloning.

1. Understand the two computers

• Windows PC: Runs WSL, Ubuntu and the compiler. You edit and compile C source here.

• Amiga 1200: Runs the resulting Motorola 68k executable under AmigaOS 3.2.

• PiStorm32Lite + Raspberry Pi 3A+: Supplies the Emu68 environment and hardware access to the I2C LCD. The demonstrated build is an AmigaOS program, not a Raspberry Pi Linux application.

• 1602 LCD + PCF8574 backpack: The working program uses `i2c.library`, a device-tree resource and a firmware mailbox interface. Hardware wiring and Amiga-side library installation are separate from WSL setup; consult the repository README.

2. Install WSL and Ubuntu on Windows

2.1 Open Windows PowerShell as administrator

Click Start, type PowerShell, right-click it and choose Run as administrator. A PowerShell window is a Windows command prompt, not yet the Linux terminal.

```
wsl --install
```

Wait for Windows to finish. Restart if prompted. The default Linux distribution is usually Ubuntu. If Ubuntu is not installed, run `wsl --list --online` and then `wsl --install -d Ubuntu`.

2.2 Start Ubuntu and create your Linux account

Open Ubuntu from the Windows Start menu. On first launch, choose a Linux username and password. The password does not show characters as you type: that is normal.
Your prompt will look something like [username]@[machine-name];

2.3 Check WSL

In PowerShell, run:

```
wsl --list --verbose
wsl --version
```

Ubuntu should appear, ideally with WSL version 2. Open Ubuntu and run:

```
cat /etc/os-release
pwd
whoami
```

The project development environment was Ubuntu 24.04.5 LTS. A newer supported Ubuntu release may also work, but package versions can differ.

3. Learn the five essential Linux commands

```
pwd            # show your current folder
ls -la         # list files, including hidden ones
cd ~           # go to your Linux home folder
mkdir -p test  # create a folder if needed
cd test        # enter that folder
```

The ~ character means your Linux home directory (for example, `/home/yourname`). Use `Ctrl+C` to interrupt a running command. Do not type the `$` or `username@computer:` prompt if a tutorial displays one.

4. Install Linux development tools

All commands in this section run in the Ubuntu terminal, not PowerShell. Copy one command block at a time, press Enter and wait until the normal prompt returns.

```
sudo apt update
sudo apt upgrade -y
```

sudo asks for your Linux password. It grants administrator permission for package installation.

```
sudo apt install -y git curl wget build-essential make
  gcc g++ flex bison gettext autoconf automake rsync
  libreadline-dev python3 python3-pip nano file
```

Check the basic tools:

```
git --version
python3 --version
make --version | head -1
```

5. Install the Motorola 68k AmigaOS cross-compiler

A normal Linux `gcc` produces programs for the PC. This project needs `m68k-amigaos-gcc`, which produces AmigaOS executables. The original integrated build used this exact compiler command name. The following installation follows the public AmigaPorts compiler project; it is a reproducible setup, not a verified record of the original toolchain installation.

5.1 Create a development tools directory

```
mkdir -p ~/tools
cd ~/tools
```

5.2 Download the compiler source

```
git clone https://github.com/AmigaPorts/m68k-amigaos-gcc.git
cd m68k-amigaos-gcc
make update
```

5.3 Prepare its installation directory

```
sudo mkdir -p /opt/amiga
sudo chown "$(id -u):$(id -g)" /opt/amiga
```

5.4 Build and install the compiler

```
make all -j"$(nproc)"
```

This can take some time and downloads additional compiler components. If a build error occurs, save the final error lines and check the upstream project instructions; do not blindly repeat destructive clean commands.

5.5 Add the compiler to your command search path

```
echo 'export PATH=/opt/amiga/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
command -v m68k-amigaos-gcc
m68k-amigaos-gcc --version
```

Expected: `command -v` prints a path ending in `m68k-amigaos-gcc`. If it does not, locate the executable with `find /opt/amiga -name m68k-amigaos-gcc` and correct your PATH. The upstream toolchain also documents explicit runtime selection; preserve the flags and runtime of a known working build when upgrading toolchains.

6. Create the LCD project workspace

These are suggested, reproducible paths. The original development used `~/pistormlcd`, and the integrated source was compiled from the home directory in WSL.

```
mkdir -p ~/pistormlcd/include
mkdir -p ~/pistormlcd/backups
mkdir -p ~/pistormlcd/build
cd ~
```

6.1 Clone the public GitHub repository

```
git clone https://github.com/mkeguber/Amiga-PiStorm32Lite-I2C-LCD-Display.git
cd ~/Amiga-PiStorm32Lite-I2C-LCD-Display
ls -la
find src -maxdepth 3 -type f | sort
```

The `find` command prints the actual filenames uploaded to GitHub. This is important because the `src` folder may contain multiple versions, headers or supporting files. Do not assume that every file in `src/` is the final integrated program.

6.2 Copy source files into the workspace

```
cp -a ~/Amiga-PiStorm32Lite-I2C-LCD-Display/src/. ~/pistormlcd/
find ~/pistormlcd -maxdepth 3 -type f | sort
```

If the repository contains an `include/` subdirectory, this command preserves it. If source and headers are stored in different locations, use the file listing to identify them. Keep the Git clone as a clean reference and edit the copies in `~/pistormlcd`.

7. Check the integrated source and its headers

The final working source was called `pistormlcd_integrated.c`. Confirm that this file exists in your workspace:

```
find ~/pistormlcd -name "pistormlcd_integrated.c"
find ~/pistormlcd -type f \( -name "*.h" -o -name "*.c" \) | sort
```

If the integrated file is under a subfolder, copy it to the workspace root, replacing the path with the one returned by `find`:

```
cp /full/path/to/pistormlcd_integrated.c ~/pistormlcd/
```

The working program used AmigaOS development headers, `i2c.library` interfaces and a device-tree resource. Some may be supplied by the cross-compiler; custom project headers belong in `~/pistormlcd/include`. Check the include statements:

```
grep -n '^#include' ~/pistormlcd/pistormlcd_integrated.c
find ~/pistormlcd/include -type f | sort
```

Important: Do not invent missing API declarations or copy arbitrary headers from unrelated repositories. If a header is absent, locate its original source or ask the repository maintainer for the exact matching version. A compiler error naming a missing header means the development setup is not yet complete.

8. Compile pistormlcd_integrated

The following is the actual successful compile command recorded during development, using the project include directory:

```
cd ~/pistormlcd
m68k-amigaos-gcc -I$HOME/pistormlcd/include \
  pistormlcd_integrated.c -o pistormlcd_integrated
```

A successful compilation returns to the prompt without errors. Confirm the output exists:

```
ls -lh ~/pistormlcd/pistormlcd_integrated
file ~/pistormlcd/pistormlcd_integrated
```

If the compiler reports an unresolved `DeviceTreeBase` symbol, the working version included a global declaration and assigned it after opening the resource:

```
APTR DeviceTreeBase = NULL;
/* Later, during initialisation: */
DeviceTreeBase = OpenResource("devicetree.resource");
```

Treat this as a diagnostic clue, not a patch to insert blindly. First compare your source with the working version and check the associated headers. The build also depends on the correct AmigaOS compiler/runtime configuration.

9. Copy the program to the Amiga and test

Copy the executable to an Amiga-readable disk or shared transfer location using your existing workflow (for example, WinUAE or a removable storage device). Do not rename the executable with a Windows `.exe` extension. On AmigaOS, ensure `i2c.library` and the relevant Emu68 support components are installed, then run the executable from AmigaShell.

The known working integrated version displays six pages, approximately two seconds each: CPU temperature and graph; temperature scale and graph; core voltage and graph; voltage scale and graph;

Fast/Chip RAM percentage free with bars; and Fast/Chip RAM free MB with bars. It does not display uptime.

Known limitation: This program runs as an AmigaOS task. During WHDLoad games that suspend multitasking, the LCD stops refreshing. Recompiling the AmigaOS application alone does not solve that limitation.

10. Troubleshooting for first-time users

• "command not found": Check you are in Ubuntu and that `/opt/amiga/bin` is on PATH. Restart Ubuntu after editing `~/.bashrc`.

• "No such file or directory": Run `pwd` and `ls`; check capitalization and the filename printed by `find`.

• Missing header: Verify the project `include/` folder and the toolchain-provided AmigaOS headers. Record the exact header name.

• Undefined symbol during linking: Save the full linker error and compare against the last working source, library and runtime configuration.

• LCD backlight but no text: This is not proof that I2C is communicating. Check Amiga-side library installation, I2C bus, backpack address, wiring and contrast separately.

• Program works in Workbench but pauses in WHDLoad: Expected limitation of the AmigaOS version, not a WSL fault.

Appendix A - Folder layout for continuing development

```
~/
  tools/
    m68k-amigaos-gcc/            # compiler source/build tree
  Amiga-PiStorm32Lite-I2C-LCD-Display/
    src/                        # downloaded GitHub source
    docs/                       # upstream project documents
  pistormlcd/
    pistormlcd_integrated.c     # editable working-source copy
    include/                    # project-specific header files
    backups/                    # dated source backups
    build/                      # optional outputs
    pistormlcd_integrated       # compiled AmigaOS program
```

Only files actually present in the GitHub repository will appear in its clone. The diagram above is a recommended local workspace, not a claim about the exact remote `src/` contents.

Appendix B - Safe AI-assisted development ("vibe coding")

Use an AI coding assistant as a collaborator, not as an authority on hardware or ABI details. Work in small, reversible steps:

1. Preserve a known working copy before edits:

```
cd ~/pistormlcd
cp pistormlcd_integrated.c \
  backups/pistormlcd_integrated_before_change.c
```

2. Describe the exact hardware and current status. For this project: Amiga 1200, PiStorm32Lite, Raspberry Pi 3A+, Emu68, HD44780 1602 LCD, PCF8574 backpack, `i2c.library` v40; backpack address `0x4E` in the working library's 8-bit convention (equivalent to 7-bit `0x27`). The LCD control bit definitions

are `RS=0x01`, `EN=0x04`, `BACKLIGHT=0x08`.

3. Give the AI the current source and full compiler error, if any. Ask for the smallest possible change and an explanation of which functions it affects.

4. Rebuild with the command in Section 8; do not change several independent things before testing.

5. Test on the actual Amiga. A successful compile does not guarantee correct I2C behaviour or display output.

6. Record what changed and whether it worked. Only then promote it to your main source or commit it to GitHub.

Suggested prompt for an AI coding assistant

```
I am modifying an existing working AmigaOS 68k C program.
Hardware: A1200, PiStorm32Lite, Pi 3A+, Emu68,
1602 LCD with PCF8574 I2C backpack.
Compiler: m68k-amigaos-gcc in Ubuntu WSL.
Preserve the existing six LCD pages and all I2C timing.
Review the source I provide. Make only the requested
change, show a unified diff, and give me one compile
command and one test at a time. Do not invent AmigaOS
APIs or change hardware addresses without evidence.
```

Appendix C - Source references

1. Project repository:

   https://github.com/mkeguber/Amiga-PiStorm32Lite-I2C-LCD-Display

2. Project source directory:

   https://github.com/mkeguber/Amiga-PiStorm32Lite-I2C-LCD-Display/tree/main/src

3. Microsoft WSL installation: https://learn.microsoft.com/windows/wsl/install

4. Microsoft WSL commands: https://learn.microsoft.com/windows/wsl/reference

5. AmigaPorts compiler: https://github.com/AmigaPorts/m68k-amigaos-gcc

Guide prepared for the public GitHub project.

Author: Michael Gruber / Sydney - Australia.